

Table Design – Sort Keys in Amazon Redshift

To Begin with the Lab

Summary of the Lab

In this lab, sort keys in **Amazon Redshift** were explored to understand how data ordering affects query performance. The automatically assigned sort key of an existing table was examined, and a new table was created using a custom compound sort key consisting of firstname and lastname columns. The assigned sort keys were then verified using Redshift system catalog views. Query execution plans were analyzed using the EXPLAIN command to understand how **Amazon Redshift** processes queries internally and estimates execution costs. Finally, the results were verified to understand how properly designed sort keys improve filtering, scanning efficiency, and overall query performance.

Understanding Sort Keys in Amazon Redshift

A sort key determines the physical order in which rows are stored on disk in **Amazon Redshift**. When queries filter, sort, or join using sort key columns, Redshift can scan fewer data blocks, reducing I/O and improving performance. Sort keys are especially useful for large analytical tables where queries frequently access data based on date ranges, customer information, or transaction details.

Example:

Consider an e-commerce Orders table containing millions of records sorted by order_date. When a query requests orders from the last 30 days, **Amazon Redshift** can quickly locate the required blocks instead of scanning the entire table. Without a sort key, every block must be scanned, resulting in slower query execution. Proper sort key selection therefore improves query efficiency and reduces processing time.

Lab Steps

- Sign in to the AWS Management Console
- Navigate to **Amazon Redshift** → open **Query Editor v2**
- Review the existing sort key configuration of the usersauto table

```
SELECT tablename,  
       "column",  
       sortkey  
FROM pg_table_def  
WHERE tablename = 'usersauto';
```

Redshift query editor v2

materialized view demo x table design demo x

Run all Isolated session Cluster or wor... dev Last saved: a few seconds ago

Run Limit 100

```

1 --- sort
2 select "column", type, sortkey
3 from pg_table_def where tablename = 'users_custom';
4
5 -- update the sort key
6 create table users_custom_sort sortkey (firstname, lastname) as
7 select * from users;
8
9 select "column", type, sortkey
10 from pg_table_def where tablename = 'users_custom_sort';

```

Result 1 (18)

column	type	sortkey
userid	integer	1
username	character(8)	0
firstname	character varying(30)	0
lastname	character varying(30)	0
city	character varying(30)	0
state	character(2)	0
email	character varying(100)	0
phone	character(14)	0

- Verify that userid is automatically assigned as the sort key
- Understand that **Amazon Redshift** automatically selects sort keys when AUTO table optimization is enabled
- Create a new table with a compound sort key

```

CREATE TABLE users_custom_sort
SORTKEY(firstname, lastname)
AS
SELECT *
FROM public.users;

```

Redshift query editor v2

materialized view demo x table design demo x

Run all Isolated session Cluster or wor... dev Last saved: a minute ago

Run Limit 100

```

4
5 -- update the sort key
6 create table users_custom_sort sortkey (firstname, lastname) as
7 select * from users;
8
9 select "column", type, sortkey
10 from pg_table_def where tablename = 'users_custom_sort';

```

Result 1

Summary

Returned rows: 0
Query ID: 3764
Elapsed time: 1.7s
Result set query:

```

/* RDEV2-WINODEUWHD */
create table users_custom_sort sortkey (firstname, lastname) as
select * from users

```

1 explain SELECT firstname, lastname, total_quantity

- Click **Run**
- Verify that the table is created successfully
- Understand that firstname and lastname form a compound sort key
- Understand that compound sort keys prioritize columns from left to right during sorting

- Understand that INTERLEAVED SORTKEY can also be used when queries filter equally on multiple columns
- Verify the assigned sort keys

```
SELECT tablename,
       "column",
       sortkey
FROM pg_table_def
WHERE tablename = 'users_custom_sort';
```

The screenshot shows the Redshift query editor v2 interface. The query editor displays the following SQL code:

```
3 from pg_table_def where tablename = 'usersauto';
4
5 -- update the sort key
6 create table users_custom_sort sortkey (firstname, lastname) as
7 select * from users;
8
9 select "column", type, sortkey
10 from pg_table_def where tablename = 'users_custom_sort';
```

The query results are displayed in a table with 3 columns: column, type, and sortkey. The results are as follows:

column	type	sortkey
userid	integer	0
username	character(8)	0
firstname	character varying(30)	1
lastname	character varying(30)	2
city	character varying(30)	0
state	character(2)	0
email	character varying(100)	0
phone	character(14)	0
likesports	boolean	0

The query ID is 3764, elapsed time is 47 ms, and total rows are 18.

- Verify that firstname is assigned sort key position 1
- Verify that lastname is assigned sort key position 2
- Analyze query performance using EXPLAIN

```
EXPLAIN
SELECT buyerid,
       SUM(qtysold)
FROM sales
GROUP BY buyerid
ORDER BY SUM(qtysold) DESC
LIMIT 10;
```

The screenshot shows the Amazon Redshift Query Editor v2 interface. The main editor displays an SQL query that has been executed. The query is as follows:

```

11 explain SELECT firstname, lastname, total_quantity
12 FROM (SELECT buyerid, sum(qtysold) total_quantity
13       FROM sales
14       GROUP BY buyerid
15       ORDER BY total_quantity desc limit 10) Q, table_design_demo.users_custom_sort U
16 WHERE Q.buyerid = U.userid
17 ORDER BY Q.total_quantity desc;

```

Below the query, the 'QUERY PLAN' section is visible, showing a tree of execution operations. The operations are highlighted with red boxes:

- XN Merge (cost=2000002006456.45..2000002006456.48 rows=11 width=27)
- Merge Key: q.total_quantity
- > XN Network (cost=2000002006456.45..2000002006456.48 rows=11 width=27)
- Send to leader
- > XN Sort (cost=2000002006456.45..2000002006456.48 rows=11 width=27)
- Sort Key: q.total_quantity
- > XN Hash Join DS_BCAST_INNER (cost=1000000005331.38..1000002006456.26 rows=11 width=27)
- Hash Cond: ("outer".userid = "inner".buyerid)
- > XN Seq Scan on users_custom_sort u (cost=0.00..499.90 rows=49990 width=23)

The bottom status bar shows 'Query ID: 3871', 'Elapsed time: 18 ms', and 'Total rows: 20'.

- Click **Run**
- Review the generated query execution plan
- Observe execution plan operations such as Merge, Network, and Sort
- Understand that EXPLAIN analyzes the execution plan without running the query
- Review important EXPLAIN metrics
 - Cost → Relative execution cost
 - Rows → Estimated rows returned
 - Width → Average row size
- Understand the Cost values returned by EXPLAIN
 - First value → Estimated cost to retrieve the first row
 - Second value → Estimated cost to return all rows
- Understand how sort keys improve query performance
 - Reduce data scanning
 - Improve filtering efficiency
 - Improve query execution performance
 - Reduce unnecessary disk reads
- Verify that firstname and lastname are successfully assigned as compound sort keys
- Verify that EXPLAIN generates a query execution plan successfully
- Verify that sort keys can be used to optimize query performance and reduce scan overhead in **Amazon Redshift**